

Lekcja 12: Mini Mario

Cel

Platformówka: poziom z liter, grawitacja, skok, kolizje ze ścianami i podłogą, kamera, monety. To most do prawdziwego Lake Mario w `src/games/mario/`.

Co nowego

Mapa kafelków. Poziom to obrazek z liter, jak sprite w lekcji 10, tylko każdy znak to kafelek 32×32 px:

```
const char* const MAP[15] = {           // 15 wierszy x 32 px = 480 px = cały ekran w pionie
    "      O      ",
    "      BBB     ",
    "#####",           // może być szersza niż ekran - stąd kamera
    "===== ",
};
load_map(MAP, "#=B");           // które znaki są STALE (nie da się przez nie przejść)
```

`map_tile(col, row)` zwraca znak, `map_set(col, row, ' ')` go zmienia (zebrana moneta znika), `map_col(px)` zamienia piksele na numer kolumny. `draw_tiles('#', DARK_GREEN, cam_x)` rysuje wszystkie kafelki `#` prostokątem, a `draw_tiles('o', sprite_monety, cam_x)` – sprite'em.

Ruch z kolizjami. Zamiast pisać samemu, czy prostokąt gracza wchodzi w kafelek, wołasz:

```
MoveResult r = move_box(p.x, p.y, PLAYER_W, PLAYER_H, p.vx, p.vy);
```

Funkcja **zmienia** `p.x`, `p.y`, `p.vx`, `p.vy` (w deklaracji ma `int&` – „przez odniesienie”): przesuwa o prędkość, ale zatrzymuje na ścianie (`vx = 0`), stawia na podłodze (`vy = 0`) i nie przepuszcza przez sufit. Zwraca strukturę `MoveResult`: `on_ground` (stoi?), `hit_wall`, `hit_head` (+ `head_col`, `head_row` – który kafelek uderzył głową). To ten sam kod, który napędza Lake Mario (`engine::TileMap`).

Kamera to po prostu liczba `cam_x` odejmowana od każdego rysowanego `x`. Gracz „stoi”, świat przesuwa się pod nim.

Uruchom

Otwórz `gra.cpp`, **Ctrl+Shift+B**. Czerwony prostokąt spada na trawę, strzałki go przesuwać, kamera podąża. Bloki i monety są na ekranie, ale skoku jeszcze nie ma – zadanie 2. Idąc w prawo wpadniesz w dziurę i spadniesz bez końca – zadanie 4.

Jak to działa

Kolejność w `frame()` jest ważna i taka sama jak w Lake Mario: **wejście** → **grawitacja** → **ruch z kolizjami** → **reakcje (monety)** → **kamera** → **rysowanie**. Grawitacja to `vy = vy + 1` co klatkę z ograniczeniem `MAX_FALL`. Kamera: `clamp(p.x - 320, 0, map_width() - 800)`.

Zadania

1. ★ Przebuduj poziom: dłuższy, więcej platform, schody z `B`. Zasada z Lake Mario: przeszkody max 3 kafelki, żeby dało się je przeskoczyć.
2. ★★ **Skok**. Przed grawitacją: `if (pressed(A) && p.on_ground) p.vy = JUMP_V;` Wysokość skoku: `13+12+11+...+1 = 91 px`, prawie 3 kafelki. Zrób skok „krótki i długi”: gdy puścisz `A` w locie (`!held(A) && p.vy < -5`), ustaw `p.vy = -5`.
3. ★★ **Monety**. Po `move_box`: `int col = map_col(p.x + PLAYER_W / 2); int row = map_row(p.y + PLAYER_H / 2);` jeśli `map_tile(col, row) == 'o'` – `map_set(col, row, ' ')` i `coins++`. (test 1)
4. ★★ **Dziura**. Gdy `p.y > map_rows() * TILE` – gracz spadł: `restart()`. Do tego licznik żyć.
5. ★★★ **Bloki do rozbicia i sprite'y**. Jeśli `r.hit_head` i `map_tile(r.head_col, r.head_row) == 'B'` – `map_set(..., ' ')` (rozbity blok). Zamiast prostokątów: bohater i moneta z lekcji 10 (`draw_tiles('o', coin, cam_x)`, `sprite(hero, p.x - cam_x, p.y, ...)`), kafelki trawy i ziemi narysuj literami po swojemu. Potem dodaj przeciwnika chodzącego po platformie (tablica struktur, `move_box` dla każdego, odbicie od ściany gdy `hit_wall`).

Sprawdź sam

Po zadaniu 3 gracz idący w prawo zbiera dwie monety leżące na trawie (kolumny 8 i 12):

```
console_sim.exe --game mini_mario --hold RIGHT 0 200 --frames 201 --trace 50
```

W ostatniej linii `coins=2`.

Dla ciekawych

Otwórz `src/games/mario/mario_game.cpp` i porównaj `update_player()` z twoim `frame()`. Znajdziesz te same kroki plus dwa triki, które sprawiają, że skok „dobrze się czuje”: bufor skoku (`JUMP_BUFFER`) i czas kojota (`COYOTE`). Przeczytaj komentarze – to prawdziwe problemy, które trzeba było zmierzyć i naprawić.

Słowniczek

po polsku	w C++
mapa z liter	<code>load_map(MAP, "#=B")</code>
znak kafelka / zmiana	<code>map_tile(col, row), map_set(col, row, ' ')</code>
piksele → kafelek	<code>map_col(px), map_row(py)</code>
ruch z kolizjami	<code>MoveResult r = move_box(x, y, w, h, vx, vy)</code>
parametr przez odniesienie (funkcja zmienia zmienną)	<code>int& x</code>
kamera	<code>cam_x, rysuj w x - cam_x</code>